

Relatório do Projeto Final : Rede De Aeroportos Do Brasil + e Rede de Faixas Musicais do Spotify + Comparação de Algoritmos

PARTE 1: Grafo de Aeroportos do Brasil (MODELAGEM)

1. NÓS: AEROPORTOS DO BRASIL

A modelagem inicial do grafo consiste na definição dos nós, que representam os aeroportos brasileiros presentes no conjunto de dados fornecidos.

Cada nó do grafo corresponde a um *aeroporto* identificado exclusivamente pelo seu código IATA (International Air Transport Association), como por exemplo: REC (Recife), GRU (São Paulo/Guarulhos) e GIG (Rio de Janeiro/Galeão).

Dessa forma, o grafo é definido como um grafo rotulado, onde:

- Cada nó representa um aeroporto;
- O rótulo do nó é o código IATA.

O conjunto de nós constitui a base estrutural do grafo, sobre a qual as interconexões (arestas) são posteriormente definidas.

2. ARESTAS

As arestas do grafo representam as conexões entre aeroportos e foram construídas a partir de critérios definidos pelo grupo, uma vez que o arquivo CSV original não contém essas informações.

Neste projeto, optou-se por considerar apenas conexões diretas reais entre aeroportos, baseadas em rotas aéreas existentes. Cada aresta representa, portanto, um voo direto entre dois aeroportos.

O grafo foi modelado como não-direcionado, ou seja, as conexões são bidirecionais. Assim, cada par de aeroportos conectados é representado por uma única aresta.

As arestas foram armazenadas no arquivo:

data/adjacencias_aeroportos.csv

Onde:

- **origem/destino:** códigos IATA dos aeroportos;
- **tipo _conexao:** classificação da conexão (ex.: regional, hub);
- **justificativa:** explicação da conexão;
- **peso:** valor numérico definido conforme a modelagem (Seção 5).

Critérios adotados:

- Conexões entre aeroportos da mesma região foram incluídas para garantir coesão regional;
- Conexões entre diferentes regiões foram incluídas para garantir conectividade global;
- Aeroportos considerados hubs possuem maior número de conexões;
- O grafo resultante é conectado, permitindo alcançar qualquer aeroporto a partir de outro.

Essa abordagem evita tanto grafos triviais (completos) quanto grafos excessivamente esparsos.

3. MÉTRICAS GLOBAIS E POR GRUPO

Este passo tem como objetivo analisar estruturalmente o grafo construído por meio de métricas globais, regionais e locais.

3.1 Métricas globais

Para o grafo completo, foram calculadas as seguintes métricas:

- **Ordem ($|V|$):** número total de aeroportos;
- **Tamanho ($|E|$):** número total de conexões;
- **Densidade:** medida de quão conectado o grafo está.

Os resultados foram armazenados no arquivo:

out/global.json

3.2 Métricas por região (subgrafos)

Foram considerados subgrafos induzidos para cada região do Brasil:

- Norte
- Nordeste
- Sudeste
- Sul
- Centro-Oeste

Para cada região, foram calculados:

- Ordem (número de aeroportos da região);
- Tamanho (número de conexões internas);
- Densidade (nível de conectividade interna).

Os resultados foram armazenados em:

out/regioes.json

3.3 Ego-redes dos aeroportos

Para cada aeroporto, foi construída sua **ego-network**, composta por:

- O próprio aeroporto;
- Todos os aeroportos diretamente conectados a ele.

Para cada ego-rede foram calculados:

- Ordem;
- Tamanho;
- Densidade.

Os resultados foram organizados na tabela:

out/ego_aeroportos.csv

com o formato:

aeroporto, grau, ordem_ego, tamanho_ego, densidade_ego

4. GRAUS E RANKINGS

O grau de um nó corresponde ao número de conexões diretas que um aeroporto possui.

Foi gerada uma lista contendo o grau de cada aeroporto, armazenada no arquivo:

out/graus.csv

no formato:

aeroporto, grau

A partir desses dados, foram identificados:

- **Aeroporto mais conectado:** aquele com maior grau, representando um hub da rede aérea;
 - Brasília (BSB) e Belo Horizonte (CNF), ambos com grau = 19
- **Aeroporto com maior densidade local:** aquele cuja ego-rede apresenta maior densidade, indicando forte interconectividade entre seus vizinhos.

- Porto Velho (PVH) e Rio Branco (RBR), ambos com densidade = 0.4

Essas métricas são fundamentais para identificar pontos estratégicos da rede, como hubs nacionais e regionais.

5. PESOS DAS ARESTAS

A modelagem do grafo da rede de aeroportos do Brasil foi realizada com base em conexões reais entre aeroportos, de modo que cada aresta representa exclusivamente voos diretos existentes. Essa abordagem busca refletir, de forma fiel, o funcionamento do sistema aéreo no contexto cotidiano.

Para a definição dos pesos das arestas, optou-se pela utilização da distância percorrida entre os aeroportos conectados.

As distâncias utilizadas foram obtidas por meio do site FlightConnections, que disponibiliza informações atualizadas sobre rotas aéreas e suas respectivas extensões. Dessa forma, o peso de cada aresta foi definido conforme a seguinte relação:

$$\text{peso} = \text{distância do voo entre os aeroportos (em quilômetros)}$$

A adoção desse modelo dispensa a aplicação de penalidades adicionais, como mudança de região ou ausência de hubs, uma vez que a distância já incorpora, ainda que indiretamente, aspectos relacionados à complexidade das rotas.

Ressalta-se que todos os pesos atribuídos são valores positivos, atendendo às exigências do algoritmo de Dijkstra, utilizado para o cálculo de caminhos mínimos, o qual não admite pesos negativos em sua execução.

Por fim, os valores correspondentes aos pesos das arestas foram armazenados no arquivo `adjacencias_aeroportos.csv`, na coluna denominada “peso”, garantindo a organização dos dados e facilitando sua utilização nos algoritmos implementados.

6. DISTANCIA ENTRE AEROPORTOS X E Y

Para determinar o menor percurso e o custo total entre os pares escolhidos de aeroportos, foi aplicado o algoritmo de Dijkstra. A escolha desse algoritmo é justificada pela modelagem adotada na Seção 5, em que todos os pesos das arestas (distâncias em quilômetros) são valores estritamente positivos, condição necessária para a correta execução do algoritmo.

Os pares de aeroportos a serem analisados foram definidos no arquivo de entrada: data/rotas.csv

Neste arquivo, foram estabelecidos pares de origem e destino, considerando:

- **Rotas obrigatórias:** Recife para Porto Alegre (REC → POA) e Manaus para São Paulo (MAO → GRU);
- **Rotas complexas adicionais:** Pares que não possuem ligação direta no grafo (ex.: Rio Branco para Florianópolis - RBR → FLN), forçando o algoritmo a calcular o caminho mínimo por meio de escalas e conexões em hubs.

Para cada par (X, Y), o algoritmo calculou o trajeto de menor custo (menor distância percorrida).

Os resultados foram consolidados e exportados para o arquivo: out/distancias_rotas.csv

O documento de saída segue a estrutura:

- **origem:** código IATA do aeroporto de partida;
- **destino:** código IATA do aeroporto de chegada;
- **custo:** somatório total das distâncias das arestas percorridas (em quilômetros);
- **caminho:** sequência exata de aeroportos (nós) percorrida do início ao fim (ex.: MAO -> BSB -> GRU).

7. ÁRVORE DE PERCURSO

Para facilitar a análise das rotas traçadas pelo algoritmo de caminhos mínimos, os percursos correspondentes aos pares obrigatórios (“Recife -> Porto Alegre” e “Manaus -> São Paulo”) foram extraídos para a formação de um subgrafo, caracterizado como uma árvore de percurso.

A visualização da árvore foi desenvolvida priorizando a interatividade e a legibilidade, utilizando a biblioteca Pyvis. O subgrafo gerado destaca as rotas por meio de codificação visual, na qual:

- Os nós correspondentes aos aeroportos visitados estão explicitamente identificados com seus respectivos rótulos (códigos IATA);
- As arestas que compõem os caminhos mínimos receberam destaque visual diferenciado (alteração de cor e espessura), facilitando o rastreamento da rota em meio à malha aérea.

O resultado gráfico interativo foi exportado e pode ser visualizado no arquivo: `out/arvore_percurso.html`

A adoção do formato HTML interativo permite a exploração fluida da estrutura da rede, viabilizando a aproximação e a manipulação dos nós para uma leitura mais clara do trajeto executado.

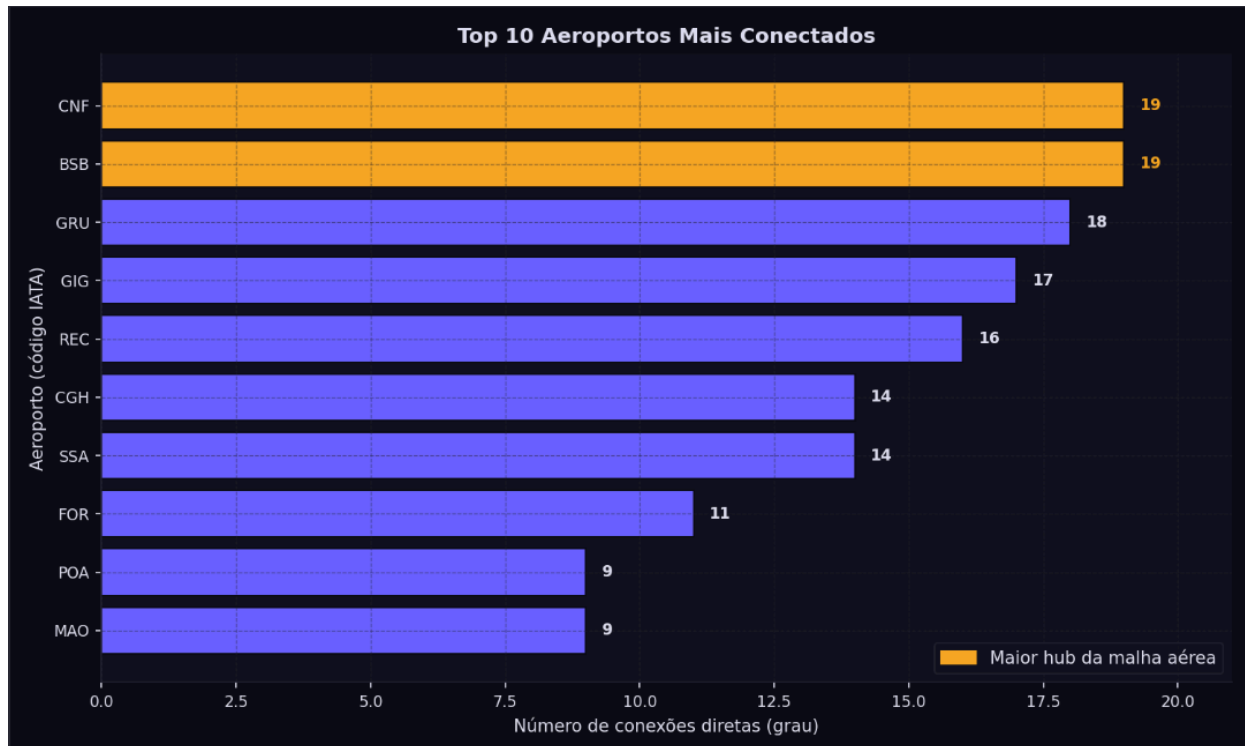
8. EXPLORAÇÕES E VISUALIZAÇÕES ANALÍTICAS

Para extrair insights a partir das métricas da rede, foram desenvolvidas quatro visualizações gráficas. A construção dessas imagens seguiu rigorosamente os princípios da disciplina de Análise e Visualização de Dados (AVD), garantindo organização estrutural, clareza na leitura e adequação do tipo de gráfico aos dados apresentados, evitando distorções visuais.

As visualizações geradas, bem como suas respectivas notas analíticas, estão detalhadas a seguir.

8.1. Ranking dos Aeroportos Mais Conectados

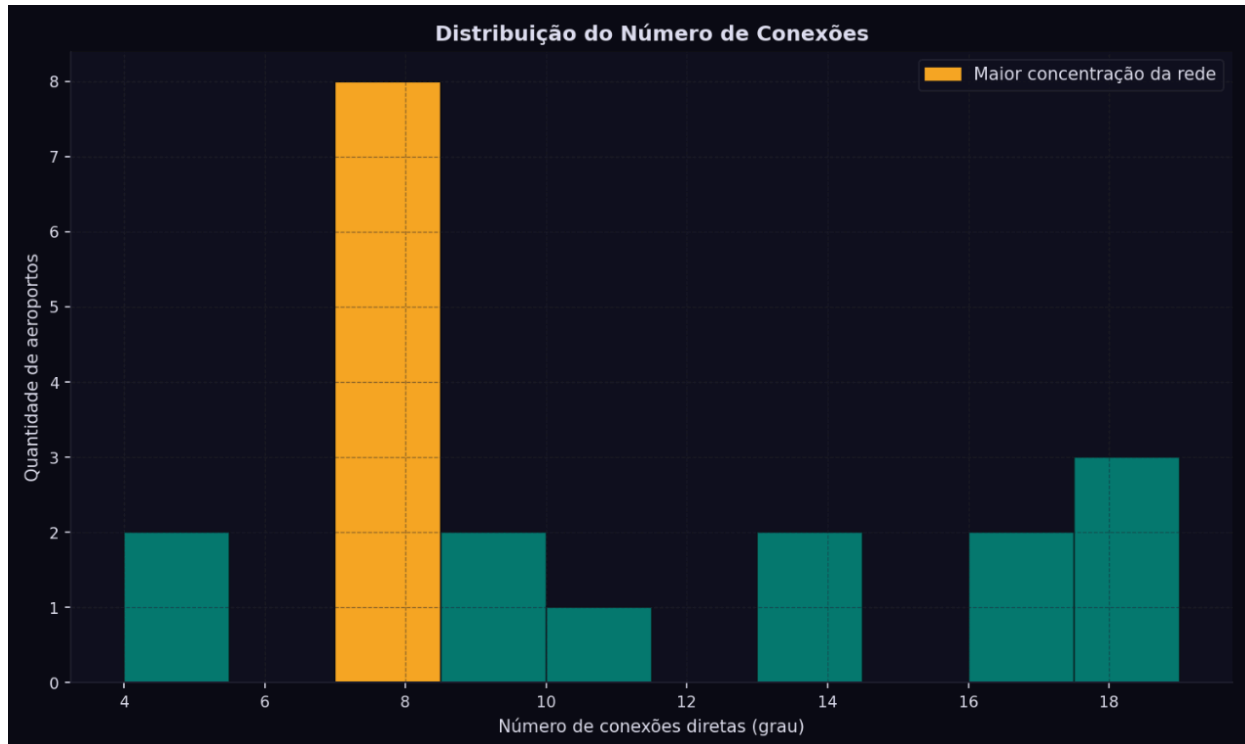
- **Arquivo de saída:** `out/vis1_ranking_hubs.png`



- **O que está sendo mostrado:** O gráfico apresenta um ranking ordenado dos aeroportos que possuem o maior número de arestas, ou seja, o maior número de voos de conexão direta na malha aérea modelada.
- **Qual insight pode ser extraído:** Por meio da visualização, é possível identificar os maiores *hubs* do país. Fica claro que os principais aeroportos concentram-se no Sudeste e Centro-Oeste (como CNF, BSB, GRU e GIG). Contudo, nota-se também a presença de outros polos regionais de extrema importância, como o aeroporto do Recife (REC), que atua como um pilar de conexão fundamental fora desse eixo.
- **Por que este tipo de visualização foi escolhido:** Optou-se pelo gráfico de barras horizontais por ser a melhor escolha para dados categóricos nominais. Essa disposição elimina a necessidade de rotacionar o texto (facilitando a leitura dos rótulos) e a ordenação decrescente cria uma hierarquia visual mais clara, garantindo maior facilidade para entender quais são os nós mais e menos conectados.

8.2. Distribuição do Número de Conexões

- Arquivo de saída: out/vis2_distribuicao_graus.png

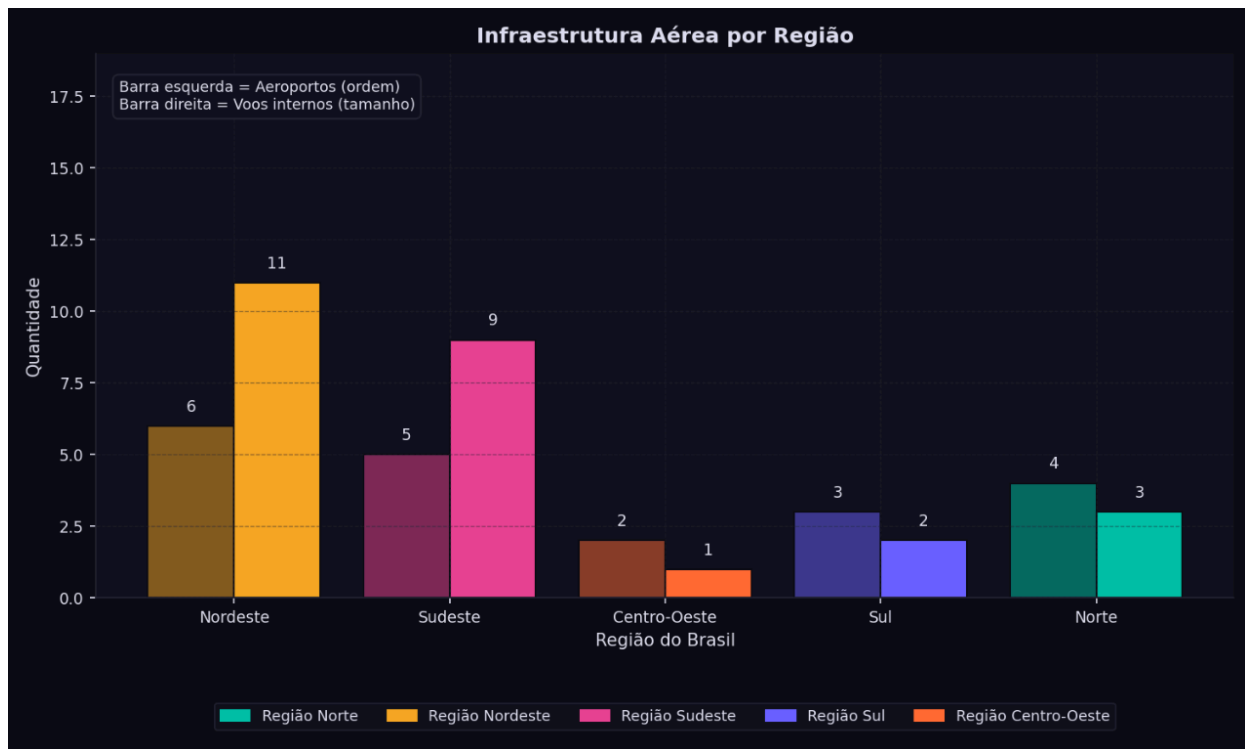


- **O que está sendo mostrado:** O gráfico exibe a frequência com que os diferentes números de conexões (graus) de cada aeroporto aparecem na malha aérea. O eixo X representa a quantidade de rotas e o eixo Y indica a quantidade de aeroportos que possuem esse volume de conexões.
- **Qual insight pode ser extraído:** Pode-se extrair o comportamento típico de redes complexas. Observa-se uma concentração muito maior de aeroportos com poucas conexões e uma parcela bem menor com uma quantidade muito alta de conexões. Estes últimos são os grandes responsáveis por sustentar e conectar a malha inteira.
- **Por que este tipo de visualização foi escolhido:** O histograma foi definido como a melhor escolha para demonstrar a distribuição de frequência de uma variável contínua ou discreta agrupada. Ele permite visualizar imediatamente o formato da distribuição dos dados (neste caso, uma notável assimetria à direita), um padrão

que seria quase impossível de perceber analisando apenas uma tabela de números soltos.

8.3. Comparativo de Infraestrutura e Voos por Região

- Arquivo de saída: out/vis3_comparacao_regioes.png

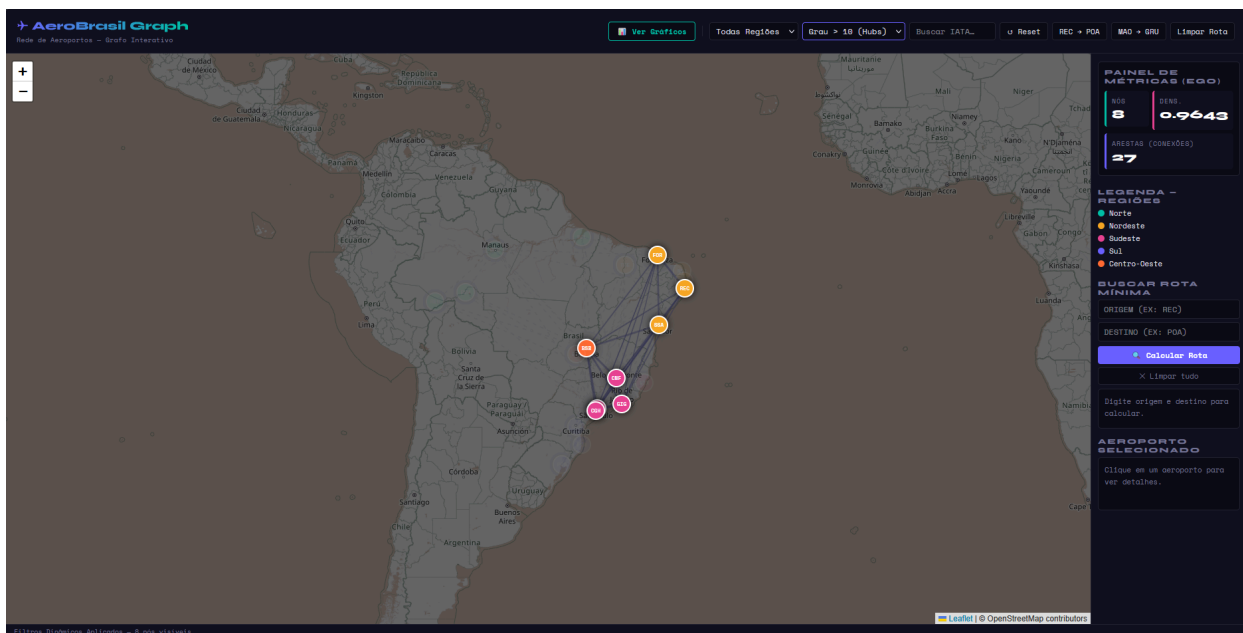


- **O que está sendo mostrado:** A visualização expõe um comparativo direto entre a quantidade de aeroportos (ordem) e a quantidade de conexões exclusivas internas (tamanho) dentro de cada região do Brasil.
- **Qual insight pode ser extraído:** O gráfico permite inferir a autossuficiência de cada região do país. Por exemplo, regiões onde a barra da direita (voos) é consideravelmente maior que a barra da esquerda (aeroportos) possuem uma malha interna densa e altamente conectada. Em contrapartida, regiões onde a barra azul é maior ou igual à laranja indicam que os aeroportos daquela área dependem mais de voos para outras regiões do que de rotas entre si mesmos.
- **Por que este tipo de visualização foi escolhido:** O gráfico de barras agrupadas é a técnica recomendada em AVD para comparar duas variáveis quantitativas (neste

caso, "ordem" e "tamanho") submetidas a uma mesma variável categórica (regiões). A escolha de cores complementares (azul e laranja) facilita a diferenciação das categorias, e a inserção de rótulos numéricos diretamente acima das barras dispensa a necessidade de o leitor consultar o eixo Y constantemente, reduzindo sua carga cognitiva.

8.4. Subgrafo Interativo dos Hubs Centrais

- Arquivo de saída: `grafo_interativo.html` via filtro



- **O que está sendo mostrado:** Uma representação topológica focada exclusivamente na "espinha dorsal" da rede. O filtro aplicado removeu os aeroportos periféricos, renderizando apenas os nós com 10 ou mais conexões e os voos que ocorrem estritamente entre eles. O tamanho de cada nó foi configurado de forma proporcional à sua quantidade de conexões no grafo total.
- **Qual insight pode ser extraído:** É possível constatar que os aeroportos mais conectados do Brasil são, também, altamente conectados entre si mesmos, formando um grande e robusto núcleo de conexões. Estruturalmente, isso significa que se um aeroporto periférico fosse desativado, a rede sofreria pouco

impacto geral; no entanto, se um desses nós centrais falhar, o impacto nacional seria severo, pois eles funcionam como pontes fundamentais de transporte.

- **Por que este tipo de visualização foi escolhido:** O grafo interativo em HTML foi eleito porque mapas estáticos contendo muitas arestas cruzadas geram alta oposição visual. A criação do subgrafo limpa e simplifica a imagem. Além disso, a codificação visual geométrica (tamanho do nó associado ao grau) atende rigorosamente aos princípios de magnitude de AVD, permitindo que o leitor identifique instantaneamente os atores mais importantes do sistema sem a obrigatoriedade de ler os rótulos.

9. Apresentação interativa do Grafo

Arquivo de saída: `out/grafico_interativo.html`

O que está sendo mostrado: A visualização consiste em uma aplicação web interativa que apresenta a malha aérea modelada sobre o mapa geográfico real do Brasil. Cada aeroporto é renderizado na sua posição geográfica exata, obtida por meio da **API do OpenStreetMap (Nominatim)**, permitindo uma leitura espacialmente coerente da rede. As conexões entre aeroportos são representadas por linhas sobre o mapa, e os caminhos obrigatórios (Recife → Porto Alegre e Manaus → São Paulo) recebem destaque visual com cores diferenciadas.

Recursos implementados:

Posicionamento geográfico real: As coordenadas de latitude e longitude de cada aeroporto foram obtidas via Nominatim e armazenadas em cache local (`data/coordenadas.json`), evitando consultas repetidas à API a cada execução. Os aeroportos são renderizados sobre o mapa do Brasil utilizando a biblioteca **Leaflet.js**, garantindo correspondência fiel entre a posição dos nós e a localização geográfica real de cada aeroporto.

Tooltip por aeroporto: Ao passar o cursor sobre qualquer aeroporto, é exibida uma janela informativa contendo o código IATA, a cidade, a região, o grau (número de conexões diretas), a ordem da ego-rede, o tamanho da ego-rede e a densidade da ego-rede, métricas extraídas diretamente do arquivo `out/ego_aeroportos.csv`.

Destaque de arestas ao clicar: Ao clicar em um aeroporto, todas as arestas diretamente conectadas a ele são imediatamente destacadas com a cor correspondente à região do aeroporto vizinho, enquanto as demais conexões são suavizadas visualmente. A barra de status exibe o número de conexões destacadas. Clicar novamente no mesmo aeroporto, ou clicar em qualquer área do mapa fora de um marcador, desfaz a seleção e restaura o estado original do grafo.

Caixa de busca: Um campo de texto permite buscar aeroportos pelo código IATA. Ao digitar, o mapa centraliza automaticamente o aeroporto correspondente com animação de zoom, e o tooltip do aeroporto é aberto automaticamente.

Botões de realce dos caminhos obrigatórios: Dois botões permitem destacar, individualmente, os caminhos mínimos calculados pelo algoritmo de Dijkstra para os pares obrigatórios. O caminho Recife → Porto Alegre é destacado em vermelho e o caminho Manaus → São Paulo em azul. As arestas que não fazem parte do caminho selecionado são escurecidas para facilitar o rastreamento da rota. Um terceiro botão restaura a visualização ao seu estado neutro.

Roteamento interativo e Filtros topológicos: A interface conta com um roteador dinâmico onde o usuário insere qualquer par de aeroportos (Origem e Destino). O algoritmo de Dijkstra calcula e traça instantaneamente o caminho mínimo no mapa. Uma barra de ferramentas superior também permite filtrar a rede exibida por Macrorregião ou por Grau de conectividade, atualizando o grafo em tempo real.

Painel lateral e Dashboard Analítico: Um painel fixo à direita exibe informações do aeroporto selecionado e um quadro dinâmico de estatísticas da rede visível atual (Ordem, Total de Arestas e Densidade). A interface possui ainda um botão de alternância que substitui o mapa por um "Dashboard" estatístico integrado (renderizado via Chart.js), apresentando gráficos interativos que detalham as propriedades matemáticas e regionais da malha.

Por que estas escolhas foram feitas: A utilização de um mapa geográfico real como base da visualização, em vez de um layout algorítmico arbitrário (como força dirigida), confere ao grafo uma dimensão analítica adicional: é possível perceber imediatamente que aeroportos geograficamente próximos tendem a ser mais conectados entre si, e que os grandes hubs nacionais estão posicionados nos centros populacionais do país. A codificação por cores

regionais permite identificar a distribuição espacial das regiões sem necessidade de consultar a legenda. O destaque interativo de arestas por clique resolve um problema clássico de visualização de grafos densos, onde o cruzamento de muitas arestas dificulta a leitura das conexões de um nó específico — ao escurecer o restante da rede, o leitor foca exclusivamente nas relações daquele aeroporto.

O algoritmo de Dijkstra é utilizado nesta etapa especificamente para calcular os dois caminhos obrigatórios exigidos pelo enunciado (Recife → Porto Alegre e Manaus → São Paulo), determinando quais arestas devem ser destacadas visualmente no mapa. Sua utilização é justificada pelo fato de todos os pesos das arestas serem valores positivos (distâncias em quilômetros), condição necessária para o funcionamento correto do algoritmo.

Implementação e Arquivos Gerados

Para a implementação da visualização interativa, o código foi distribuído entre dois arquivos, seguindo rigorosamente a estrutura de pastas obrigatória do projeto. O arquivo principal **src/viz.py** é responsável por toda a lógica de leitura dos dados, obtenção das coordenadas geográficas e geração do HTML interativo. O algoritmo de Dijkstra, por sua vez, não foi implementado diretamente neste arquivo — ele é importado do módulo **src/graphs/algorithms.py**, que centraliza todas as implementações de algoritmos de grafos do projeto, mantendo a separação de responsabilidades e evitando duplicação de código.

Durante a execução do script, além do arquivo de saída principal, foi necessária a criação de um arquivo auxiliar não previsto inicialmente na estrutura do projeto:

data/coordenadas.json — arquivo de cache gerado automaticamente na primeira execução do script. Armazena as coordenadas geográficas (latitude e longitude) de cada aeroporto obtidas via API do OpenStreetMap (Nominatim). Nas execuções seguintes, o script lê as coordenadas diretamente deste arquivo, evitando novas consultas à API e tornando a geração do HTML significativamente mais rápida.

Bibliotecas Locais — pasta lib/

Durante o desenvolvimento da visualização interativa, identificou-se a necessidade de garantir que o arquivo **out/grafico_interativo.html** funcionasse corretamente mesmo sem acesso à internet, uma vez que versões anteriores do script carregavam as bibliotecas JavaScript diretamente de servidores externos (CDN). Para resolver essa dependência, as bibliotecas foram baixadas e armazenadas localmente na pasta `lib/`, na raiz do projeto. A seguir, a descrição de cada biblioteca e sua função no projeto:

vis-9.1.2/ — Vis.js Network: É a biblioteca principal da visualização. Responsável por renderizar o grafo de aeroportos na tela, gerenciando a criação dos nós (aeroportos), das arestas (conexões), os eventos de clique, zoom, arrastar e todas as interações visuais com o grafo. O arquivo **vis-network.min.js** é a versão compactada (minificada) da biblioteca, otimizada para carregamento rápido. A pasta `vis-network/` contém os arquivos auxiliares necessários para seu funcionamento.

tom-select/ — Tom Select: Biblioteca responsável pela caixa de busca interativa da interface. Transforma o campo de texto simples em um componente com autocomplete, permitindo que o usuário comece a digitar um código IATA e receba sugestões em tempo real dos aeroportos disponíveis. O arquivo **tom-select.complete.min.js** contém a lógica do componente e o `tom-select.css` define sua aparência visual.

bindings/utils.js — Utilitários de integração: Arquivo auxiliar que faz a ponte entre a biblioteca Vis.js e os demais componentes da página. Contém funções utilitárias de suporte que garantem o funcionamento correto dos eventos e interações entre o grafo e os elementos da interface, como o painel lateral e os botões de destaque de rotas.

10. Análise exploratória e explanatória dos dados (AVD)

Para encerrar a análise do grafo de aeroportos, foram produzidas quatro visualizações com foco em extrair e comunicar os principais padrões estruturais da rede. O código responsável pela geração de todas as imagens está em `src/analise_etapa10.py`, e os arquivos de saída foram salvos na pasta `out/`.

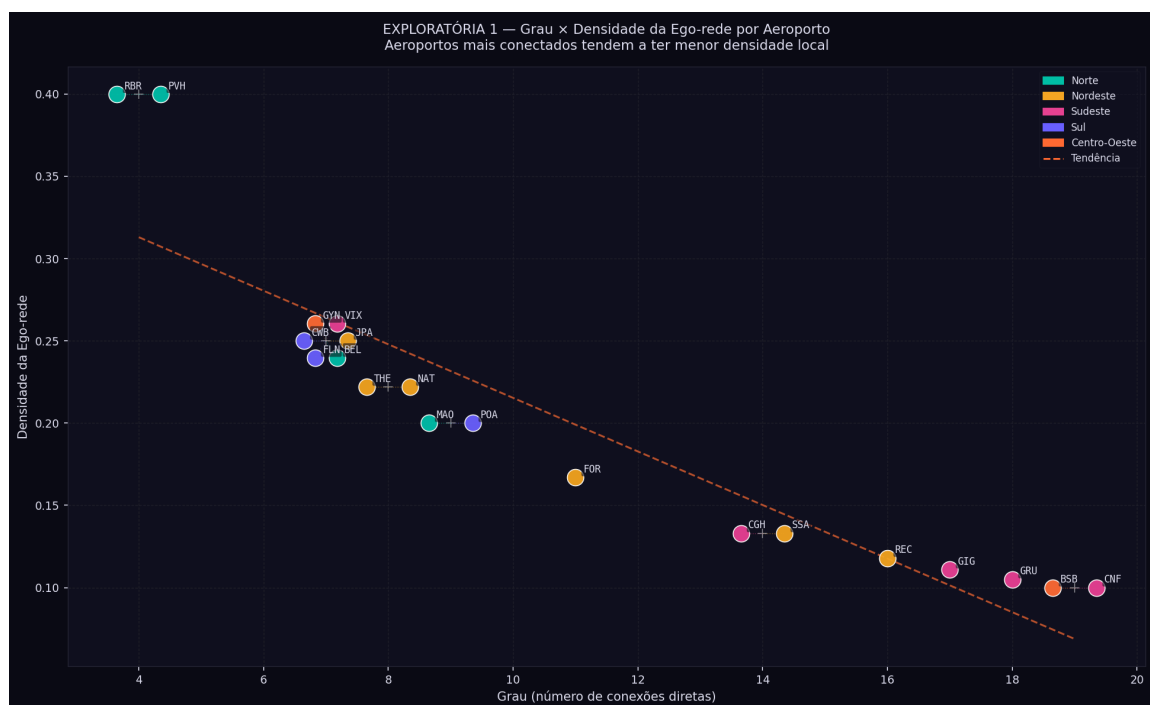
Exploração das métricas

Antes de partir para as visualizações, os dados foram analisados a partir de três fontes principais: **out/graus.csv**, **out/ego_aeroportos.csv** e **out/regioes.json**. A análise revelou os seguintes padrões gerais:

Os graus variam de 4 (PVH e RBR, no Norte) a 19 (CNF e BSB, no Sudeste e Centro-Oeste), indicando uma rede fortemente assimétrica onde poucos aeroportos concentram a maioria das conexões. A densidade global do grafo é relativamente alta dado o número de nós, reflexo da modelagem baseada em rotas reais.

A densidade das ego-redes apresenta comportamento inverso ao grau: aeroportos com mais conexões (como CNF e BSB, densidade_ego = 0.1) têm vizinhanças menos densas, pois seus vizinhos nem sempre se conectam entre si. Já aeroportos periféricos como PVH e RBR (densidade_ego = 0.4) têm vizinhanças menores e mais coesas.

10.1. Visualização Exploratória 1 — Grau × Densidade da Ego-rede



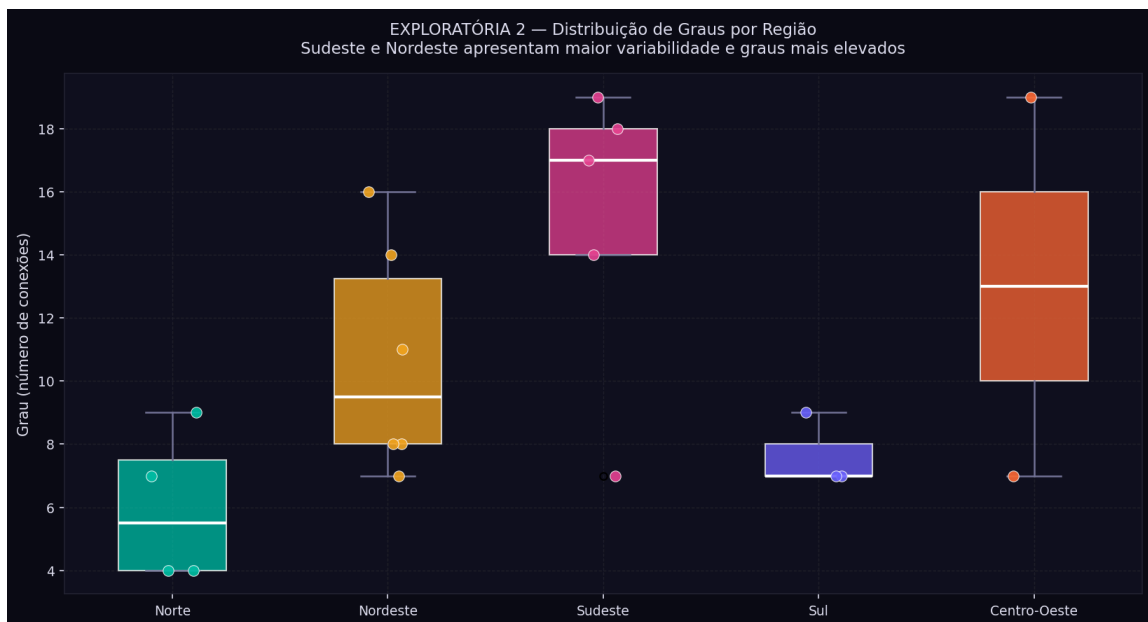
Arquivo de saída: **out/vis_exp1_dispersao_grau_densidade.png**

O que está sendo mostrado: Gráfico de dispersão onde cada ponto representa um aeroporto. O eixo X indica o grau (número de conexões diretas) e o eixo Y indica a densidade da ego-rede. Os pontos são coloridos por região e uma linha de tendência evidencia a correlação entre as variáveis. Aeroportos com posições sobrepostas são deslocados em arco com linhas tracejadas indicando a posição real.

Qual insight pode ser extraído: A correlação negativa entre grau e densidade local é clara — quanto mais conectado o aeroporto, menos densa é sua vizinhança. Isso confirma o funcionamento hub-and-spoke da malha aérea brasileira: os grandes hubs centralizam conexões com aeroportos periféricos que não se conectam entre si, enquanto os aeroportos menores operam em grupos regionais mais coesos.

Por que este tipo de visualização foi escolhido: O gráfico de dispersão é a ferramenta mais adequada para investigar a relação entre duas variáveis quantitativas contínuas. A adição da linha de tendência transforma a visualização de exploratória em confirmação de hipótese, e a codificação por cor regional permite observar se o padrão se repete de forma uniforme entre as regiões.

10.2. Visualização Exploratória 2 — Distribuição de Graus por Região



Arquivo de saída: `out/vis_exp2_boxplot_graus_regiao.png`

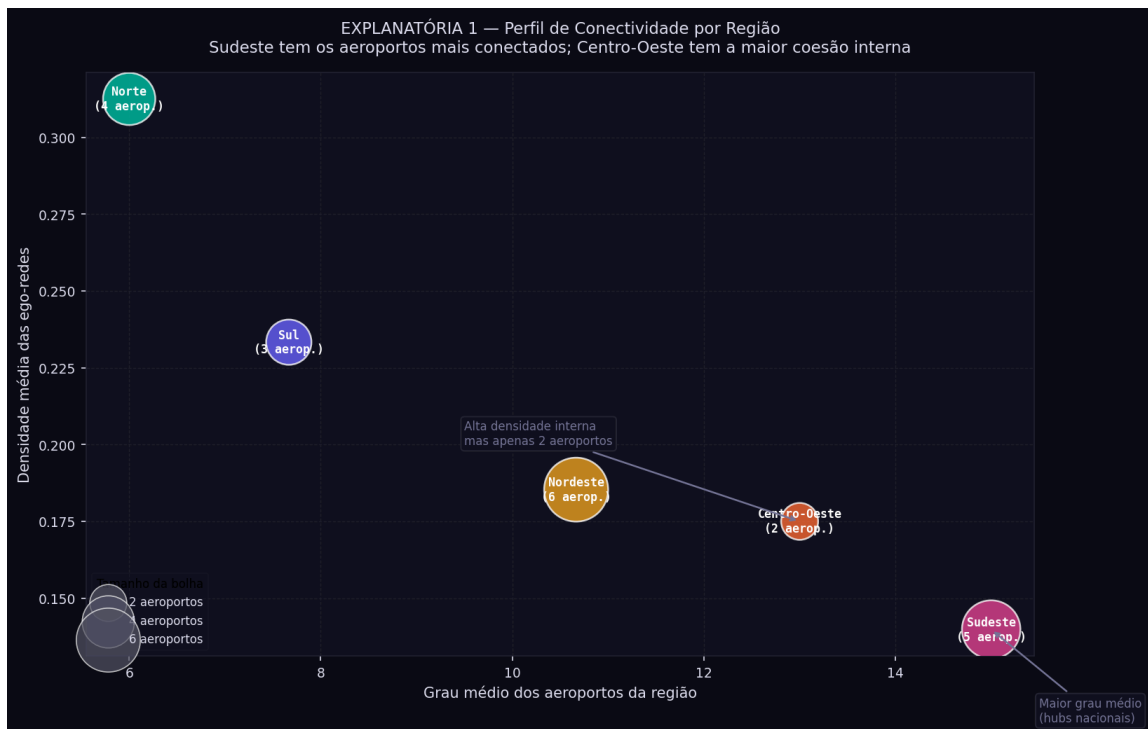
O que está sendo mostrado: Boxplot comparativo dos graus dos aeroportos agrupados por região, com os pontos individuais sobrepostos para revelar a distribuição real dos dados dentro de cada caixa.

Qual insight pode ser extraído: O Sudeste apresenta a maior mediana de graus e também a maior variabilidade interna — reflexo da coexistência de megahubs (CNF com 19, GIG com 17) e

aeroportos com conectividade moderada (VIX com 7). O Norte é a região com menor grau mediano e menor variação, indicando uma malha mais uniforme e periférica. O Centro-Oeste, apesar de ter BSB como um dos hubs mais conectados, tem apenas dois aeroportos, o que limita a interpretação estatística do boxplot para essa região.

Por que este tipo de visualização foi escolhido: O boxplot é a representação mais eficiente para comparar distribuições entre grupos categóricos. A sobreposição dos pontos individuais elimina a perda de informação típica do boxplot puro, permitindo ao leitor ver simultaneamente o resumo estatístico e os dados brutos.

10.3. Visualização Explanatória 1 — Perfil de Conectividade por Região



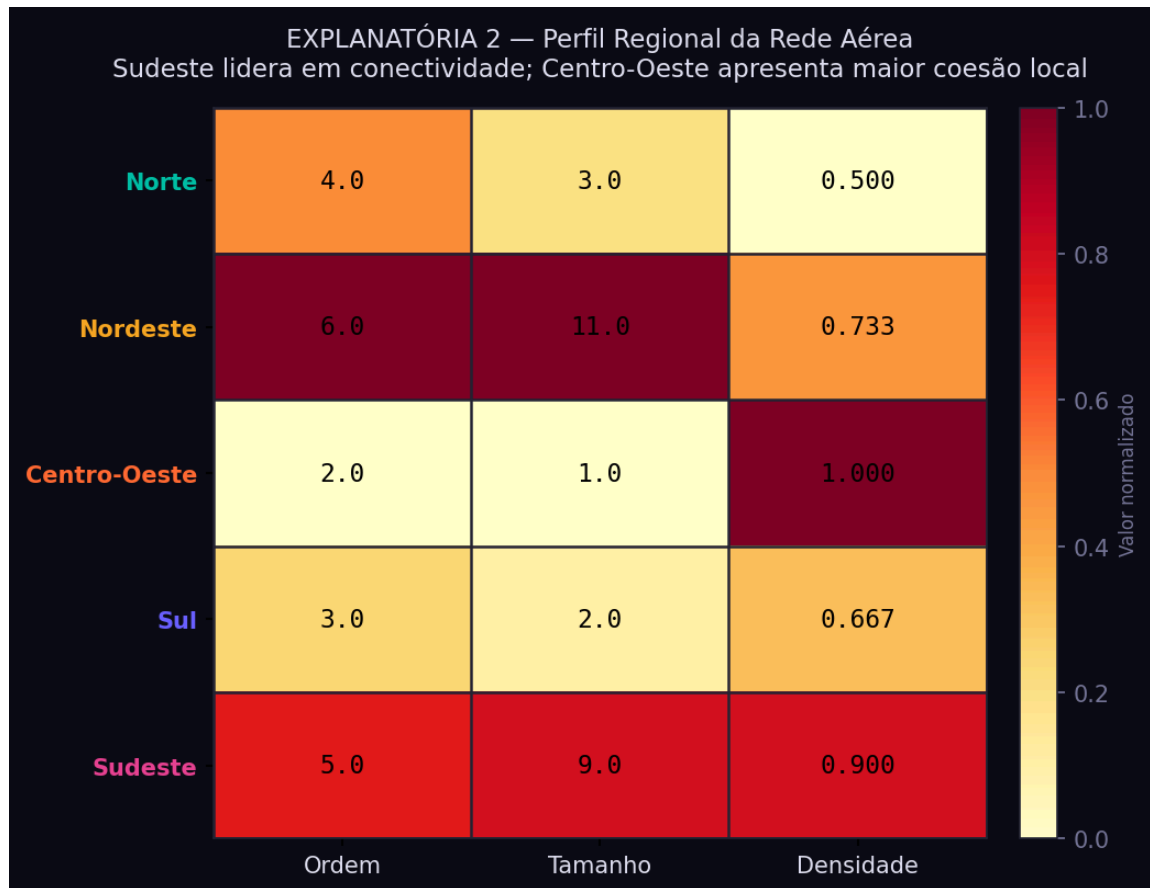
Arquivo de saída: `out/vis_expl1_bolhas_regioes.png`

O que está sendo mostrado: Gráfico de bolhas onde cada bolha representa uma região brasileira. O eixo X indica o grau médio dos aeroportos da região, o eixo Y indica a densidade média das ego-redes, e o tamanho de cada bolha é proporcional ao número de aeroportos da região. Anotações explicativas destacam os casos mais relevantes.

Qual insight pode ser extraído: O Norte se destaca com a maior densidade média de ego-rede, mesmo sendo uma região com aeroportos de grau médio baixo, isso indica que seus poucos aeroportos operam em grupos regionais muito coesos entre si. O Sudeste ocupa o extremo oposto: maior grau médio de todas as regiões (acima de 14), mas com a menor densidade de ego-rede, confirmando o comportamento típico de hub — aeroportos altamente conectados cujos vizinhos não necessariamente se conectam entre si. O Nordeste, com 6 aeroportos (maior bolha), posiciona-se no centro do gráfico, representando o perfil mais equilibrado entre conectividade e coesão local. O Sul apresenta grau médio e densidade intermediários, com 3 aeroportos. O Centro-Oeste, apesar de ter apenas 2 aeroportos e grau médio elevado (próximo de 13), apresenta densidade de ego-rede relativamente baixa, reflexo de BSB operar como hub nacional com vizinhança pouco coesa internamente.

Por que este tipo de visualização foi escolhido: O gráfico de bolhas é indicado quando se deseja comunicar simultaneamente três variáveis quantitativas e uma categórica. Sua leitura é intuitiva para qualquer pessoa, mesmo sem conhecimento prévio do projeto, pois o tamanho, posição e cor das bolhas transmitem a mensagem principal sem necessidade de consultar tabelas. As anotações com setas direcionam o olhar do leitor para os casos mais relevantes, seguindo boas práticas de design analítico explanatório.

10.4. Visualização Explanatória 2 — Perfil Comparativo das Regiões Brasileiras



Arquivo de saída: out/vis_expl2_mini_heatmap_regioes.png

O que está sendo mostrado:

Mapa de calor simplificado (mini heatmap) comparando as cinco regiões brasileiras em três métricas estruturais da rede:

- **Ordem** → quantidade de aeroportos da região;
- **Tamanho** → número total de conexões/arestas internas da sub-rede regional;
- **Densidade** → nível de conectividade interna entre os aeroportos da região.

Cada linha representa uma região e cada coluna representa uma métrica. As células são coloridas em escala contínua do amarelo claro ao vermelho escuro, onde cores mais intensas indicam valores relativamente maiores após normalização dos dados entre 0 e 1. Os valores reais são exibidos dentro de cada célula para preservar a precisão quantitativa da análise.

Qual insight pode ser extraído:

O heatmap evidencia rapidamente os contrastes estruturais entre as regiões brasileiras. O Nordeste aparece como a região com maior ordem (6 aeroportos) e maior tamanho interno (11 conexões), indicando a sub-rede regional mais ampla da malha. O Sudeste, apesar de possuir menos aeroportos (5), apresenta densidade extremamente elevada (0.9), mostrando que seus aeroportos estão fortemente conectados entre si. O Centro-Oeste se destaca por possuir densidade máxima (1.0), resultado da conexão completa entre seus dois aeroportos. Já o Norte apresenta densidade significativamente menor (0.5), sugerindo uma malha regional mais dispersa e dependente de conexões externas. O Sul ocupa posição intermediária em todas as métricas, caracterizando uma estrutura regional mais equilibrada.

Por que este tipo de visualização foi escolhido:

O heatmap foi escolhido por permitir comparação simultânea de múltiplas métricas entre poucos grupos categóricos de forma extremamente compacta e legível. Diferentemente do heatmap anterior por aeroporto — que possuía alta carga visual e excesso de informação — esta versão agregada reduz a complexidade e enfatiza apenas os padrões regionais mais relevantes. A codificação por intensidade de cor facilita a identificação imediata das regiões dominantes em cada dimensão, enquanto os valores numéricos preservam a precisão analítica necessária para interpretação acadêmica. O resultado é uma visualização explanatória mais limpa, objetiva e adequada para síntese comparativa da estrutura da rede aérea brasileira.

Síntese da Análise

A análise conjunta das quatro visualizações permite concluir que a malha aérea brasileira modelada apresenta características típicas de uma rede de mundo pequeno com estrutura hub-and-spoke: poucos aeroportos altamente conectados (CNF, BSB, GIG, GRU, REC) sustentam a conectividade global da rede, enquanto aeroportos periféricos dependem desses hubs para alcançar qualquer destino. Regionalmente, o Sudeste e o Centro-Oeste concentram os nós mais estratégicos, enquanto o Norte apresenta a maior vulnerabilidade estrutural — a remoção de um único hub nessa região poderia isolar aeroportos como PVH e RBR do restante da malha nacional.

PARTE 2: Grafo do Dataset Spotify (MODELAGEM E COMPARAÇÃO DE ALGORITMOS)

1. DATASET E MOTIVAÇÃO

O dataset utilizado na Parte 2 é o Spotify Tracks Dataset, disponível publicamente na plataforma Kaggle. Ele reúne informações sobre 89.740 faixas musicais.

O arquivo principal do dataset foi armazenado em:

data/dataset_parte2/dataset.csv

2. MODELAGEM DO GRAFO

2.1 Nós: Faixas Musicais, gêneros e artistas

Cada nó do grafo representa uma faixa musical, identificada de forma única pelo seu `track_id`.

Como o dataset completo possui quase 90 mil faixas e a construção de uma matriz de distâncias para esse volume seria inviável, foi realizada uma **amostragem por gênero musical**. A amostra resultante contém **1.921 faixas**, distribuídas de forma proporcional entre os gêneros presentes no dataset.

2.2 Arestas:

As arestas foram construídas com base na **distância euclidiana entre os vetores de atributos de áudio** de cada par de faixas. Para cada faixa, foram conectadas as **K = 30 faixas mais similares**, formando um grafo KNN.

O grafo resultante é **dirigido e ponderado**, com as seguintes características:

- **Tipo:** dirigido, ponderado
- **Nós:** 1.921 faixas
- **Arestas:** 57.630 conexões
- **Grau de saída:** K = 30 para todos os nós
- **Grau médio:** 30,0

2.3 Features de Áudio Utilizadas

- Os vetores de cada faixa são compostos pelas seguintes 9 features numéricas:

danceability	Adequação da faixa para dança
energy	Intensidade e atividade perceptual
loudness	Volume médio em decibéis
speechiness	Presença de palavras faladas
acousticness	Probabilidade de a faixa ser acústica
instrumentalness	Ausência de vocais
liveness	Probabilidade de performance ao vivo
valence	Positividade musical transmitida
tempo	Tempo em BPM (batidas por minuto)

2.4 Peso das Arestas

O peso de cada aresta corresponde à **distância euclidiana normalizada** entre os vetores de áudio dos dois nós conectados, conforme a fórmula:

$$\sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

Todos os pesos do grafo principal são estritamente não-negativos. Para a demonstração do Bellman-Ford com pesos negativos, foi construído um subgrafo auxiliar com offset negativo.

3. ALGORITMOS IMPLEMENTADOS

Todos os quatro algoritmos foram implementados do zero em `src/graphs/algorithms.py`, sem o uso de bibliotecas externas de grafos como NetworkX ou iGraph. A estrutura do grafo é definida em `src/graphs/graph.py` com lista de adjacência.

Algoritmo	Complexidade	Suporta pesos negativos
BFS	$O(V + E)$	Não se aplica
DFS	$O(V + E)$	Não se aplica
Dijkstra	$O((V + E) \log V)$	Não
Bellman-Ford	$O(V \cdot E)$	Sim, detecta ciclo negativo

4. EXECUÇÃO DOS ALGORITMOS E RESULTADOS

Todos os quatro algoritmos foram implementados do zero em `src/graphs/algorithms.py`, sem o uso de bibliotecas externas de grafos como NetworkX ou iGraph. A estrutura do grafo é definida em `src/graphs/graph.py` com lista de adjacência.

4.1 BFS: Busca em Largura

O BFS foi executado a partir de **3 fontes distintas**, escolhidas aleatoriamente entre as 1.921 faixas da amostra. Em todas as execuções, o algoritmo percorreu a totalidade dos nós do grafo, confirmando que o grafo é **fortemente conexo** a partir de qualquer vértice.

4.2 DFS: Busca em Profundidade

O DFS foi executado a partir de **3 fontes distintas**, com classificação completa de arestas em quatro categorias: tree, back, forward e cross.

A detecção de ciclos em todas as execuções era esperada, pois, em um grafo KNN dirigido, é altamente provável que os vizinhos de uma faixa também apontem de volta para ela.

4.3 Dijkstra: Caminho Mínimo com Pesos Positivos

O Dijkstra foi executado para 5 pares de faixas, usando o grafo principal com pesos positivos. O algoritmo encontrou caminhos mínimos válidos em todos os casos.

4.4 Bellman-Ford: Pesos Negativos e Detecção de Ciclo

Para demonstrar as capacidades do Bellman-Ford além do cenário coberto pelo Dijkstra, foram construídos dois subgrafos especiais.

Cenário 1: Pesos negativos sem ciclo negativo (DAG)

Foi construído um **grafo acíclico dirigido (DAG)** com 300 nós e 1.147 arestas, extraído da amostra principal.

O offset aplicado foi de $-(\text{media_dist} / 2)$, tornando **1.036 das 1.147 arestas negativas**. A estrutura DAG (arestas apenas de índice menor para maior) garante ausência de ciclos e, portanto, a corretude do Bellman-Ford.

Cenário 2: Detecção de ciclo negativo

Foi injetado artificialmente um ciclo negativo $a \rightarrow b \rightarrow c \rightarrow a$ com peso $-1,0$ em cada aresta (total $-3,0$ no ciclo). O Bellman-Ford detectou corretamente a situação na fase de verificação final, interrompendo o processamento e sinalizando `ciclo_negativo_detectado = true`.

A detecção é quase instantânea, e isso ocorre porque o ciclo negativo é identificado na etapa de verificação, antes de qualquer tentativa de reconstrução do caminho.

5. COMPARAÇÃO DE DESEMPENHO ENTRE ALGORITMOS

Os tempos médios de execução de cada algoritmo foram medidos com `time.perf_counter()` e mostrados no arquivo `out/parte2_report.json`.

Algoritmo	Tempo Médio (s)	Grafo utilizado
BFS	0,006	Principal
DFS	0,027	Principal
Dijkstra	0,005	Principal
Bellman-Ford (sem ciclo neg.)	0,0007	DAG auxiliar
Bellman-Ford (ciclo neg.)	0,00004	Subgrafo artificial

Análise comparativa:

O **DFS** foi o mais lento entre os algoritmos executados no grafo principal, pois além de percorrer todos os nós e arestas, realiza a classificação individual de cada aresta.

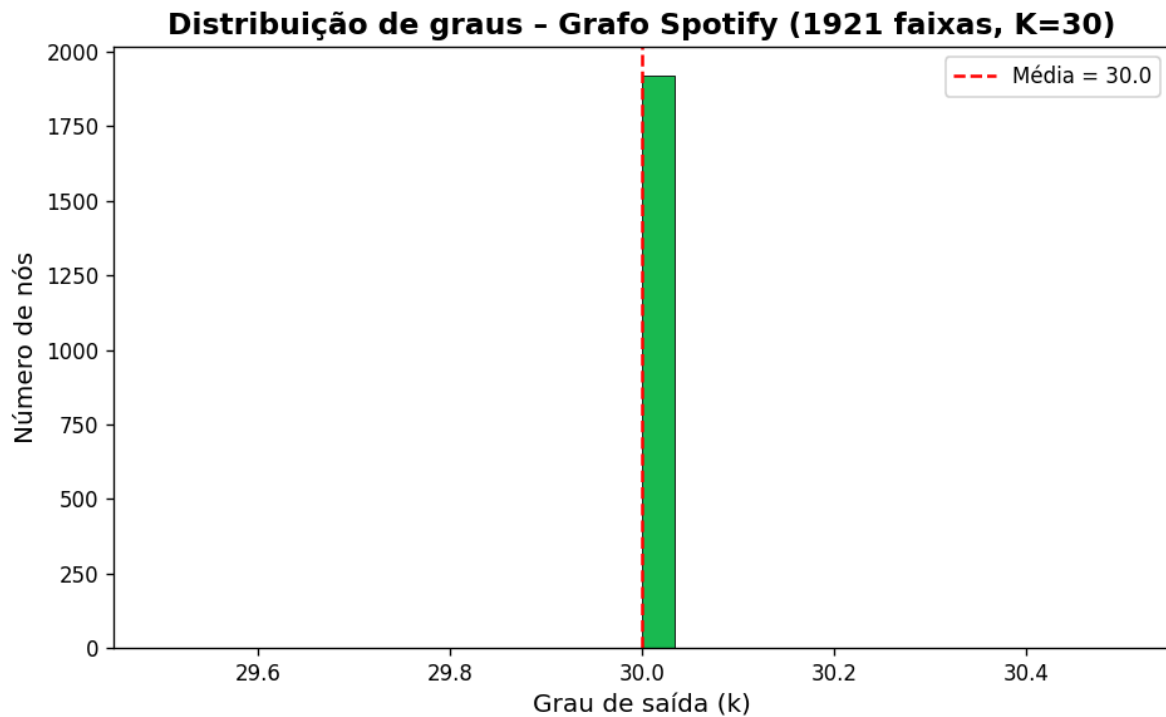
O **BFS** e o **Dijkstra** apresentaram tempos muito próximos. Isso é esperado, pois, em um grafo com pesos positivos e estrutura uniforme como o KNN, a fila de prioridade do Dijkstra não acrescenta custo significativo em relação à fila simples do BFS.

O **Bellman-Ford** foi executado em um subgrafo menor (300 nós), o que explica seus tempos reduzidos. Em teoria, aplicado ao grafo principal, seria ordens de magnitude mais lento que o Dijkstra.

6. VISUALIZAÇÕES GERADAS

6.1 VISUALIZAÇÕES GERADAS

Arquivo: out/p2_vis1_distribuicao_graus.png



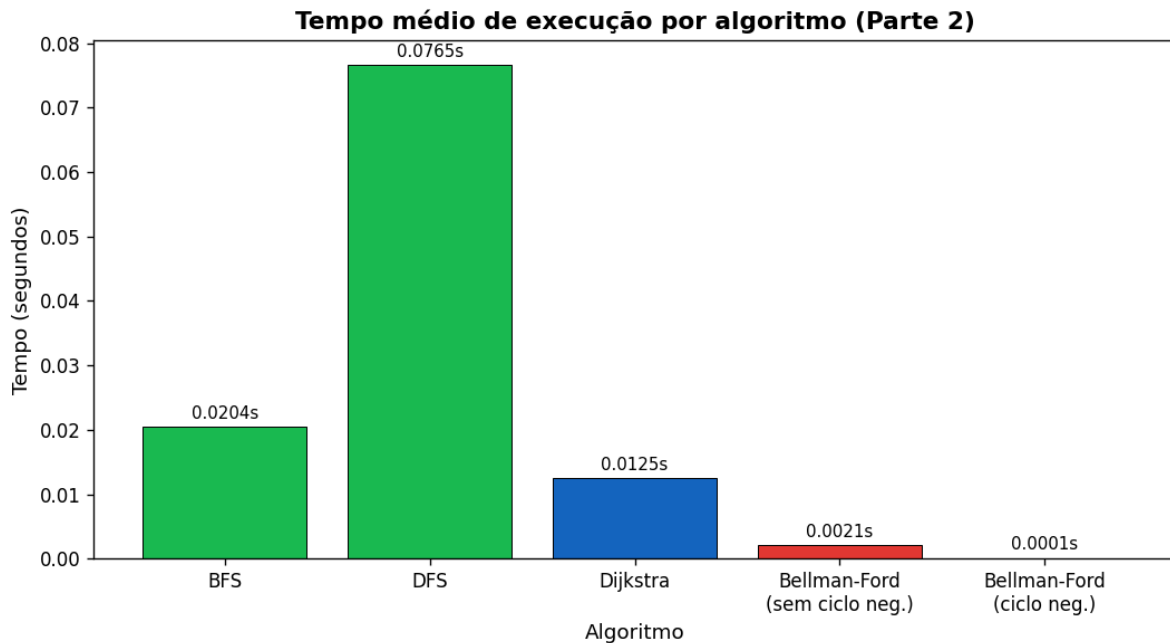
O que está sendo mostrado: Histograma da distribuição dos graus de saída dos nós no grafo KNN principal. Uma linha vertical em vermelho indica o grau médio.

Qual insight pode ser extraído: Por construção KNN com K fixo, todos os nós possuem exatamente grau de saída 30, resultando em um histograma com spike único. Isso diferencia fundamentalmente esse grafo do de aeroportos (Parte 1), onde a distribuição era assimétrica. Essa uniformidade de saída não implica uniformidade de entrada, diferentes faixas podem ser vizinhas de diferentes quantidades de outras faixas, gerando variação nos graus de entrada.

Por que este tipo de visualização foi escolhido: O histograma é o formato canônico para distribuição de frequências em uma variável discreta. Mesmo no caso trivial do grafo KNN, a visualização confirma a propriedade de construção e serve como verificação de sanidade do pipeline.

6.2 Comparação de Tempo Médio por Algoritmo

Arquivo: out/p2_vis2_tempo_algoritmos.png



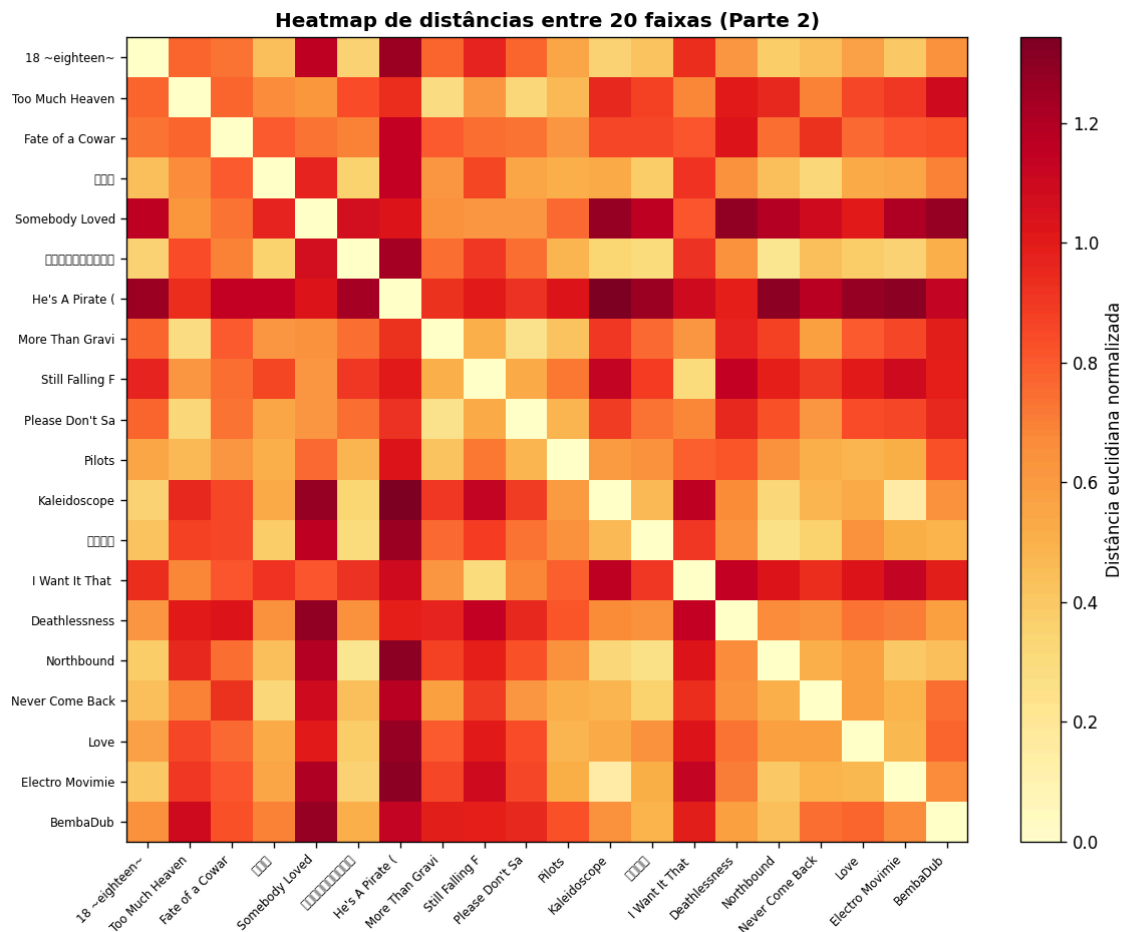
O que está sendo mostrado: Gráfico de barras com o tempo médio de execução de cada um dos cinco cenários de algoritmo (BFS, DFS, Dijkstra, Bellman-Ford sem ciclo negativo e Bellman-Ford com ciclo negativo). Os valores são exibidos diretamente sobre cada barra.

Qual insight pode ser extraído: A diferença de escala entre os algoritmos é imediatamente visível. O DFS é o mais lento no grafo principal, seguido de BFS e Dijkstra com tempos muito similares. A separação visual entre os dois cenários de Bellman-Ford evidencia que a detecção de ciclo negativo é virtualmente instantânea, enquanto a execução completa no DAG, embora rápida, já supera os algoritmos de caminho mínimo aplicados ao grafo principal, o que antecipa o comportamento proibitivo do BF em grafos de larga escala.

Por que este tipo de visualização foi escolhido: O gráfico de barras é a representação mais direta para comparar magnitudes de uma variável quantitativa entre categorias discretas. A codificação por cor diferencia visualmente os algoritmos de travessia (verde) dos algoritmos de caminho mínimo (azul e vermelho), facilitando agrupamento conceitual sem necessidade de legenda adicional.

6.3 Heatmap de Distâncias entre Faixas

Arquivo: out/p2_vis3_heatmap_distancias.png



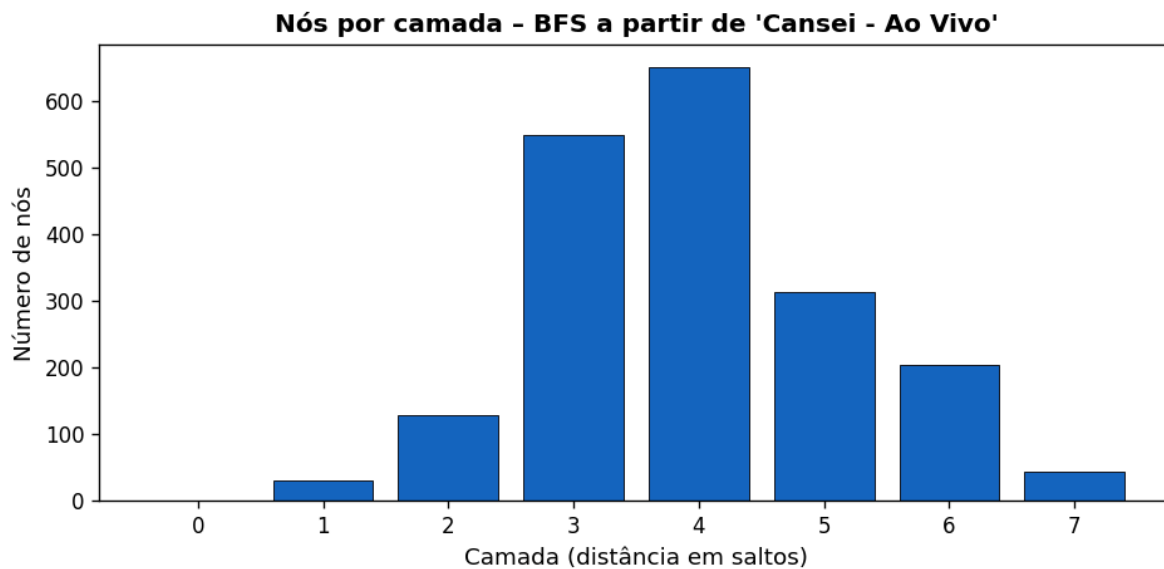
O que está sendo mostrado: Mapa de calor da matriz de distâncias normalizadas entre as 20 primeiras faixas do subgrafo do Bellman-Ford. Cada célula (i, j) representa a distância entre a faixa i e a faixa j no espaço de features de áudio.

Qual insight pode ser extraído: Células com tons mais frios (baixa distância) indicam faixas com perfis de áudio muito similares. Células com tons quentes (alta distância) representam faixas acusticamente distantes. A diagonal principal é sempre zero.

Por que este tipo de visualização foi escolhido: O heatmap é a representação ideal para matrizes de distância ou correlação, pois permite identificar padrões de proximidade e agrupamento de forma intuitiva, sem necessitar de coordenadas explícitas. A paleta YlOrRd (amarelo-laranja-vermelho) segue convenções de visualização de distâncias, onde cores mais quentes indicam maior separação.

6.4 Distribuição de Nós por Camada (BFS)

Arquivo: out/p2_vis4_bfs_camadas.png



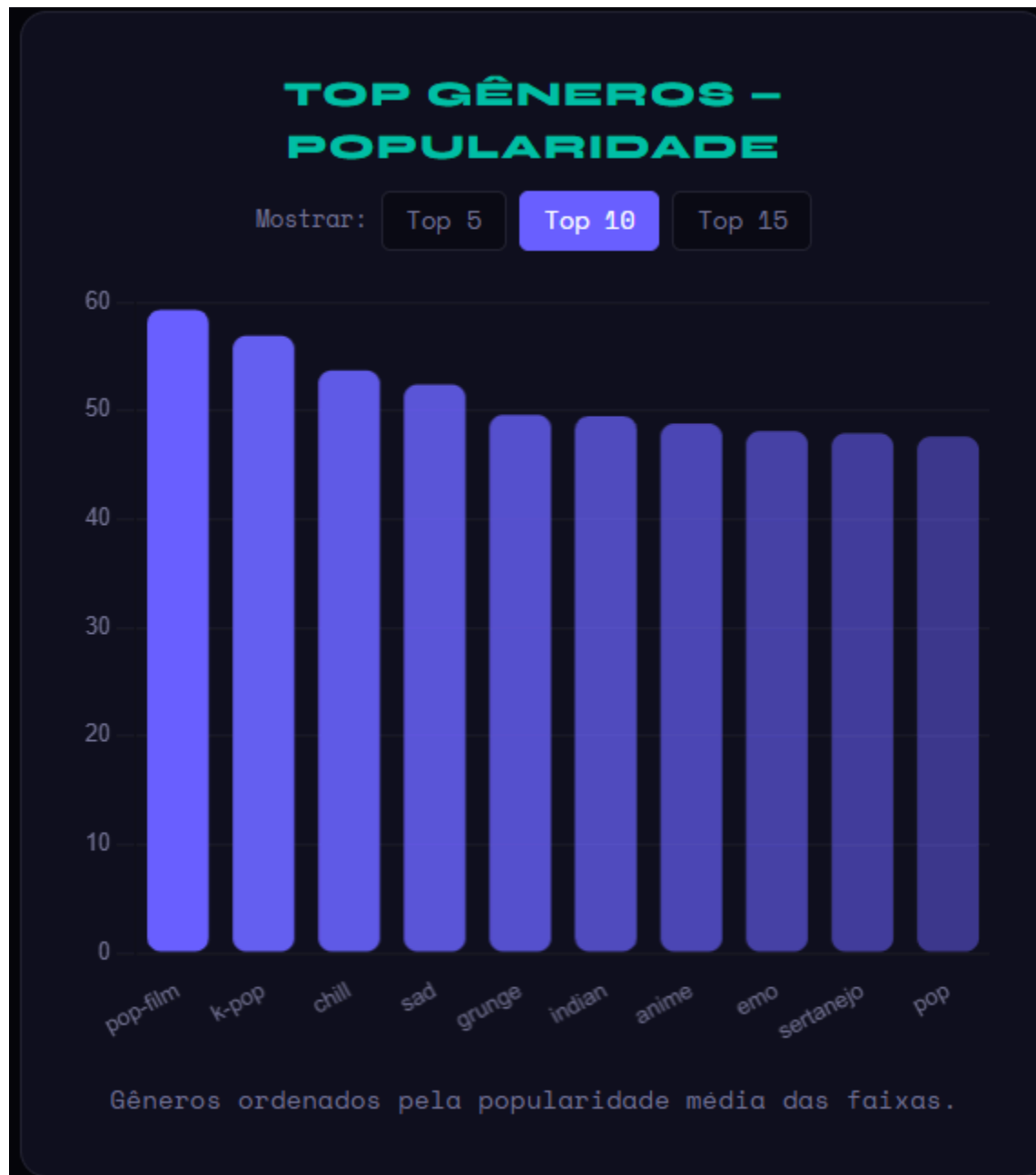
O que está sendo mostrado: Histograma com o número de nós em cada camada de distância a partir da faixa de origem do primeiro BFS executado ("Cansei - Ao Vivo"). O eixo X representa o número de saltos a partir da origem e o eixo Y indica quantos nós se encontram a essa distância.

Qual insight pode ser extraído: A maioria dos nós está concentrada nas camadas intermediárias (camadas 3 a 5), e não na primeira ou última. Isso é característico de redes densas com pequeno diâmetro: a partir de qualquer nó, a "onda de expansão" do BFS cresce rapidamente nas primeiras camadas (cada nó tem 30 vizinhos imediatos), atinge seu pico nas camadas do meio e se esgota rapidamente. A existência de apenas 7 a 9 camadas para cobrir quase 1.920 nós confirma a propriedade de mundo pequeno identificada na Seção 4.1.

Por que este tipo de visualização foi escolhido: O gráfico de barras por camada é a representação natural do resultado do BFS, pois a execução do algoritmo já organiza os nós em camadas de forma intrínseca. A visualização torna imediato o que uma tabela de nós e distâncias nunca conseguiria, a forma da distribuição de alcançabilidade em função da distância em saltos.

Gráficos Dinâmicos

6.5 Top Gêneros por Popularidade



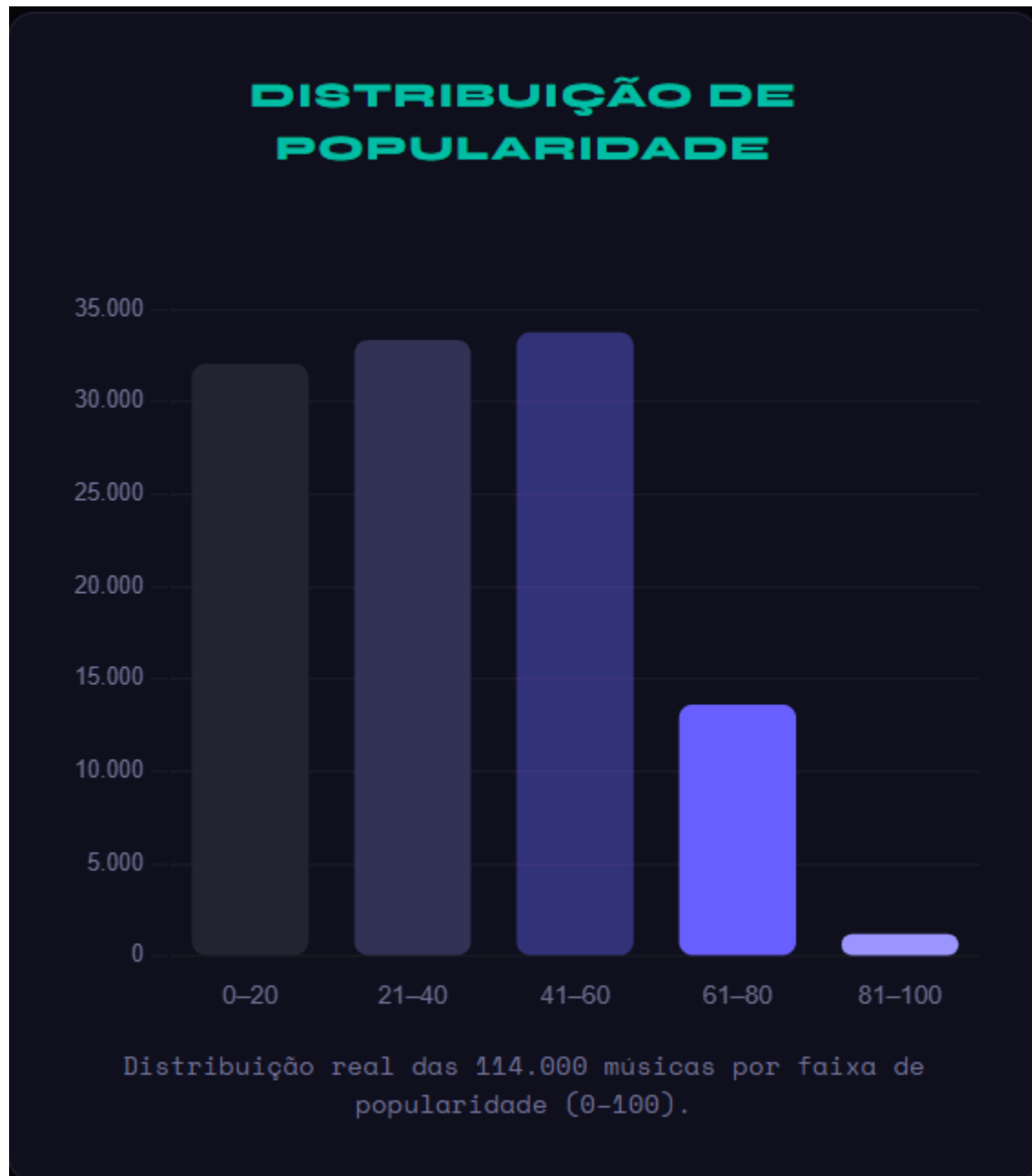
O que está sendo mostrado: Gráfico de barras horizontais exibindo os gêneros musicais com maior popularidade média das faixas no dataset Spotify, com filtro interativo para Top 5, Top 10 ou Top 15 gêneros.

Qual insight pode ser extraído: O gênero pop-film lidera com popularidade média próxima de 58, seguido de k-pop e chill, indicando que gêneros associados a trilhas sonoras e entretenimento visual ou de

nicho jovem dominam o ranking. A diferença entre o primeiro e o décimo colocado é relativamente pequena (≈ 10 pontos), sugerindo que os gêneros mais populares formam um grupo compacto, sem um outlier dominante absoluto. A presença de gêneros como sertanejo e indian entre os Top 10 evidencia a diversidade geográfica e cultural da base de usuários.

Por que este tipo de visualização foi escolhido: O gráfico de barras é ideal para comparação de valores em categorias nominais como gêneros musicais. O filtro Top 5/10/15 permite ao usuário controlar o nível de detalhe sem sobrecarregar a visualização, e a ordenação decrescente facilita a leitura imediata do ranking.

6.6 Distribuição de Popularidade das Faixas



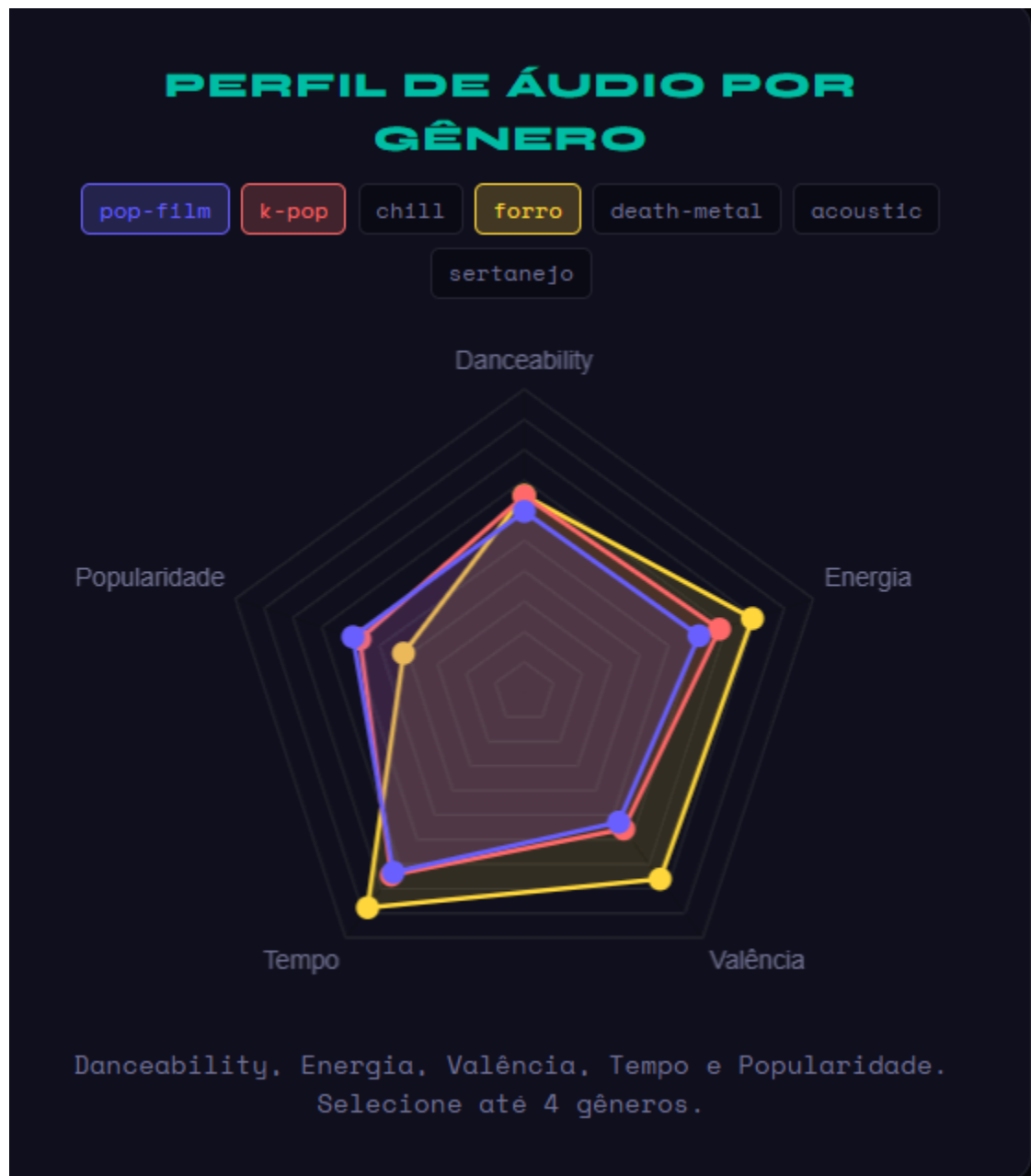
O que está sendo mostrado: Histograma agrupado em faixas de 20 pontos (0-20, 21-40, 41-60, 61-80, 81-100) representando a distribuição de popularidade das 114.000 músicas do dataset Spotify.

Qual insight pode ser extraído: A distribuição é assimétrica à direita (right-skewed): a grande maioria das faixas se concentra nas faixas de baixa e média popularidade (0-60), com os três primeiros intervalos somando mais de 95.000 músicas. Faixas com popularidade acima de 60 são minoria, e acima de 80 são raríssimas, evidenciando que músicas genuinamente populares são exceção no catálogo. Isso é esperado

em plataformas de streaming, onde o catálogo é vasto mas a atenção dos usuários se concentra em poucos hits — fenômeno típico da cauda longa.

Por que este tipo de visualização foi escolhido: O histograma com bins de amplitude igual é o formato mais direto para revelar a forma de uma distribuição contínua discretizada. A escolha de 5 intervalos equilibra granularidade e legibilidade, permitindo identificar rapidamente onde a massa de dados se concentra sem fragmentar excessivamente o eixo.

6.7 Perfil de Áudio por Gênero

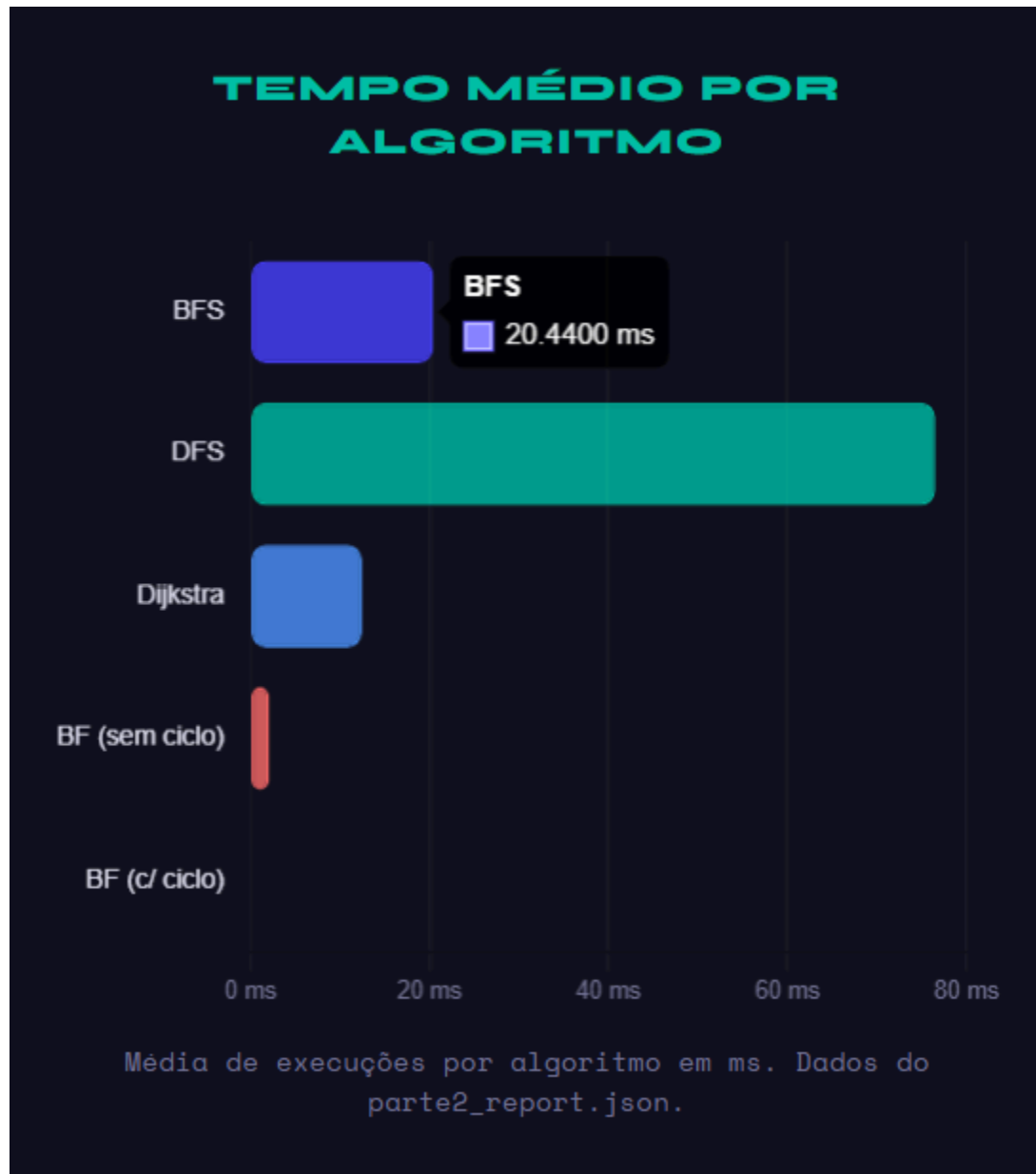


O que está sendo mostrado: Gráfico de radar (spider chart) com cinco dimensões de áudio — Danceability, Energia, Valência, Tempo e Popularidade — comparando até 4 gêneros simultaneamente. Cada polígono representa o perfil médio de um gênero, com seleção interativa via botões.

Qual insight pode ser extraído: Gêneros como forró e k-pop apresentam perfis expandidos em Danceability e Valência, indicando músicas mais dançantes e com tom emocional mais positivo. Já death-metal, quando selecionado, tende a se destacar em Energia mas retrair em Valência, revelando o padrão clássico de gêneros intensos porém melancólicos ou agressivos. A sobreposição dos polígonos permite identificar quais dimensões realmente diferenciam os gêneros — enquanto Tempo e Energia variam bastante entre estilos, Popularidade tende a se manter em faixa similar para os gêneros mais ouvidos.

Por que este tipo de visualização foi escolhido: O radar chart é especialmente adequado para comparar múltiplos atributos quantitativos de categorias distintas de forma simultânea, tornando imediata a percepção de "formato" de cada gênero. A interatividade com seleção de até 4 gêneros evita poluição visual e permite comparações direcionadas pelo usuário.

6.8 Tempo Médio por Algoritmo



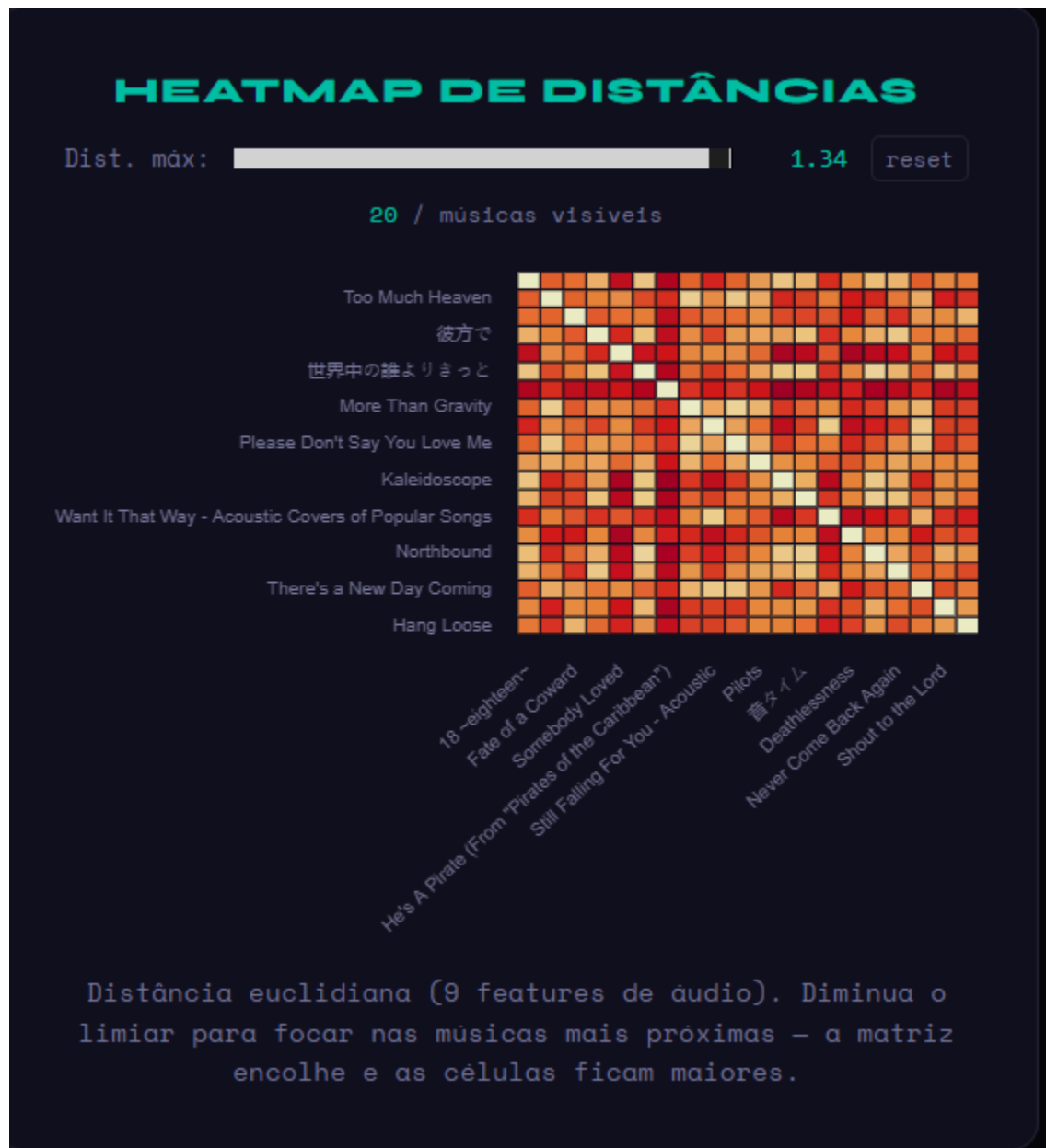
O que está sendo mostrado: Gráfico de barras horizontais com o tempo médio de execução (em ms) de cada algoritmo de busca/caminho aplicado ao grafo Spotify: BFS, DFS, Dijkstra, Bellman-Ford sem ciclo e Bellman-Ford com ciclo, com dados extraídos do `parte2_report.json`.

Qual insight pode ser extraído: O DFS é disparado o algoritmo mais lento, com tempo médio próximo de 80ms, enquanto o Bellman-Ford com detecção de ciclo praticamente não registra tempo mensurável — provavelmente por abortar a execução antecipadamente ao detectar a condição de parada. O BFS (≈ 20 ms) e o Dijkstra (≈ 15 ms) ficam em patamar intermediário, com Dijkstra ligeiramente mais eficiente apesar de computacionalmente mais pesado em teoria, o que pode refletir o impacto da estrutura do grafo KNN nas

travessias. O Bellman-Ford sem ciclo apresenta tempo muito baixo, sugerindo que os caminhos testados são curtos ou que o grafo favorece convergência rápida.

Por que este tipo de visualização foi escolhido: O gráfico de barras horizontais facilita a leitura dos rótulos de algoritmos no eixo vertical e permite comparação imediata de magnitude entre os tempos. A escala em ms e o tooltip interativo (visível no BFS) tornam os valores exatos acessíveis sem poluir o gráfico com anotações estáticas.

6.9 Heatmap de Distâncias entre Faixas



O que está sendo mostrado: Matriz de calor exibindo a distância euclidiana (calculada sobre 9 features de áudio) entre pares de faixas musicais. O slider de distância máxima filtra quais músicas aparecem na visualização, e o número de músicas visíveis se ajusta dinamicamente conforme o limiar escolhido.

Qual insight pode ser extraído: A presença de células claras (distância baixa) e escuras (distância alta) na mesma matriz indica que, mesmo entre as faixas selecionadas, há grande heterogeneidade de proximidade sonora. Faixas como Too Much Heaven e 彼方で formam blocos de baixa distância entre si, sugerindo perfis de áudio similares, enquanto outras combinações resultam em células vermelhas intensas, apontando dissimilaridade elevada. A estrutura não é diagonal dominante uniforme, o que

confirma que a proximidade no espaço de features não é trivialmente transitiva — duas músicas próximas de uma terceira não são necessariamente próximas entre si.

Por que este tipo de visualização foi escolhido: O heatmap é a representação canônica para matrizes de similaridade/distância, permitindo identificar clusters visuais e outliers de forma imediata. O slider interativo de limiar é especialmente útil aqui pois reduz a matriz ao subconjunto mais relevante, evitando o problema de legibilidade que surgiria ao tentar exibir todas as 114.000 faixas simultaneamente.

6.10 Nós por Camada – BFS



O que está sendo mostrado: Histograma da distribuição dos 1.920 nós visitados pelo BFS a partir da faixa de origem Cansei – Ao Vivo, agrupados por camada de distância em saltos (0 a 7).

Qual insight pode ser extraído: A distribuição segue um padrão de sino assimétrico, com crescimento rápido nas camadas iniciais, pico na Camada 4 (≈ 640 nós) e decaimento gradual até a Camada 7. Isso indica que o grafo possui boa conectividade local — a partir de um único nó, o BFS alcança centenas de vizinhos em apenas 3–4 saltos — mas que a expansão desacelera nas camadas mais profundas, sugerindo que o componente visitado tem diâmetro efetivo em torno de 7 saltos. A Camada 0 contém apenas o nó de origem e a Camada 1 poucos vizinhos diretos, reflexo do K fixo do grafo KNN ($K=30$).

Por que este tipo de visualização foi escolhido: O histograma por camada é a forma mais direta de revelar a estrutura de expansão do BFS, evidenciando como a "fronteira" de busca cresce e encolhe à medida que o algoritmo avança. Essa visualização complementa diretamente a análise do grafo KNN e serve como evidência empírica do diâmetro e da densidade de conectividade do grafo.

7. TESTES UNITÁRIOS

7.1 BFS — 5 testes

O foco principal foi garantir que o algoritmo respeita a ordem de largura. Os testes verificam se os níveis (camadas) estão corretos, se um nó pai sempre é visitado antes dos seus filhos, se o mapa de predecessores é montado certo e se nós isolados (sem nenhuma aresta) são ignorados na varredura. O último teste usa um grafo linear simples $A \rightarrow B \rightarrow C \rightarrow D$ para confirmar que as profundidades batem com o esperado.

7.2 DFS (tests/test_dfs.py) - 6 testes

Os testes cobrem as duas responsabilidades principais do DFS: detectar ciclos e classificar arestas. Foram criados grafos com e sem ciclos para verificar que não há falsos positivos nem falsos negativos. Há testes específicos para confirmar que arestas de retorno (back edges) aparecem em grafos cíclicos e que arestas da árvore DFS são rotuladas corretamente. Um teste a parte verifica que um triângulo não-dirigido também é detectado como cíclico.

7.3 Dijkstra (tests/test_dijkstra.py) - 6 testes

Além do caso básico de caminho correto, os testes verificam situações de borda:

1. origem=destino (custo zero);
 2. nó inalcançável (retorna infinito e caminho vazio)
 3. grafo com rota alternativa mais longa para confirmar que o algoritmo prefere o caminho mais barato.
- O teste mais importante é o que confirma que o algoritmo lança `ValueError` ao receber uma aresta com peso negativo, já que isso é uma restrição fundamental do Dijkstra.

7.4 Bellman-Ford (tests/test_bellman_ford.py) - 5 testes

Os testes exploram exatamente o que diferencia o Bellman-Ford dos outros: suporte a pesos negativos e detecção de ciclo negativo. Há um teste que confirma que o custo ótimo passa por uma aresta de peso -2 em vez de tomar o caminho positivo mais curto. A detecção de ciclo negativo é testada com o ciclo $B \rightarrow C \rightarrow B$ de peso total -2 .

Por fim, um teste verifica que quando os pesos são todos positivos, o Bellman-Ford produz exatamente o mesmo resultado que o Dijkstra, confirmando que os dois são equivalentes nesse cenário.

8. GERAÇÃO DE PLAYLISTS:

8.1 BFS:

Ao selecionar a opção em um nó de criar playlist com BFS, o sistema irá usar o algoritmo de BFS para pegar os 15 nós mais próximos à raiz, formando uma playlist que, no geral, possui músicas/artistas muito similares ao escolhido

8.2 DFS:

Ao selecionar a opção em um nó de criar playlist com DFS, o sistema irá usar o algoritmo de DFS para, gradualmente, escolher a música/artista mais parecida com a anterior, visto que faz uma busca em profundidade. Assim, se forma uma experiência profunda, onde quem for ouvir a playlist pode terminar com uma música gradualmente cada vez mais diferente da primeira escolhida.

9. ARQUIVOS GERADOS PELA PARTE 2

1. out/parte2_report.json (métricas do dataset, tempos e resultados dos algoritmos)
2. out/p2_vis1_distribuicao_graus.png (histograma de graus do grafo)
3. out/p2_vis2_tempo_algoritmos.png (comparação de tempo médio por algoritmo)
4. out/p2_vis3_heatmap_distancias.png (heatmap de distancias entre o top 20 faixas)
5. out/p2_vis4_bfs_camadas.png (distribuição de nós por camada)
6. Playlists_geradas.txt (listas de playlists geradas pelas árvores)

10. SÍNTESE E CONCLUSÕES

A Parte 2 do projeto demonstrou, de forma prática, como um dataset sem relações explícitas (o catálogo do Spotify) pode ser convertido em um grafo significativo por meio de uma métrica de similaridade bem definida, neste caso, a distância euclidiana no espaço de features de áudio.

O grafo KNN resultante apresenta características de rede de mundo pequeno: altamente conectado, com diâmetro reduzido (7 a 9 camadas para ~ 1.920 nós) e ciclos abundantes. Essas propriedades tornam o dataset ideal para estressar algoritmos de travessia e caminho mínimo em condições realistas.

A comparação entre os quatro algoritmos revelou trade-offs claros entre universalidade e eficiência. O Dijkstra é uma ótima escolha para grafos com pesos positivos, combinando velocidade e corretude. O BFS é igualmente rápido quando a ponderação das arestas é irrelevante. O DFS, embora indispensável

para detecção de ciclos e classificação de arestas, paga um custo adicional por essas funcionalidades. O Bellman-Ford, mais poderoso por suportar pesos negativos e detectar ciclos negativos, é proibitivamente lento em grafos de larga escala, seu valor prático reside em cenários específicos onde os outros algoritmos falham.

Por fim, a implementação de todos os algoritmos do zero, acompanhada de uma suíte de testes unitários com 22 casos cobrindo comportamentos corretos, casos extremos e falhas esperadas, garante a correção das implementações independentemente do dataset utilizado.